

AI X CITY CLIMATE ACTION HACKATHON 2026

FloodFact AI

Live Product Demonstration

Every screen in this file is a real screenshot of the running application, captured with Playwright — not a mockup.

Repository: github.com/VinceOdundo/FloodfactAI

Team: Samuel Mburu Kairu · Vincent Odundo · Alvin Nyota · Amani Maria — Nairobi, Kenya

Two independent layers of realism, both honestly labeled

Whole-app demo mode

No Supabase project is configured for this deck. DEMO_MODE drives every screen from the same seed narrative as supabase/seed.sql — the identical query functions that read live Postgres through RLS in production instead read static fixtures.

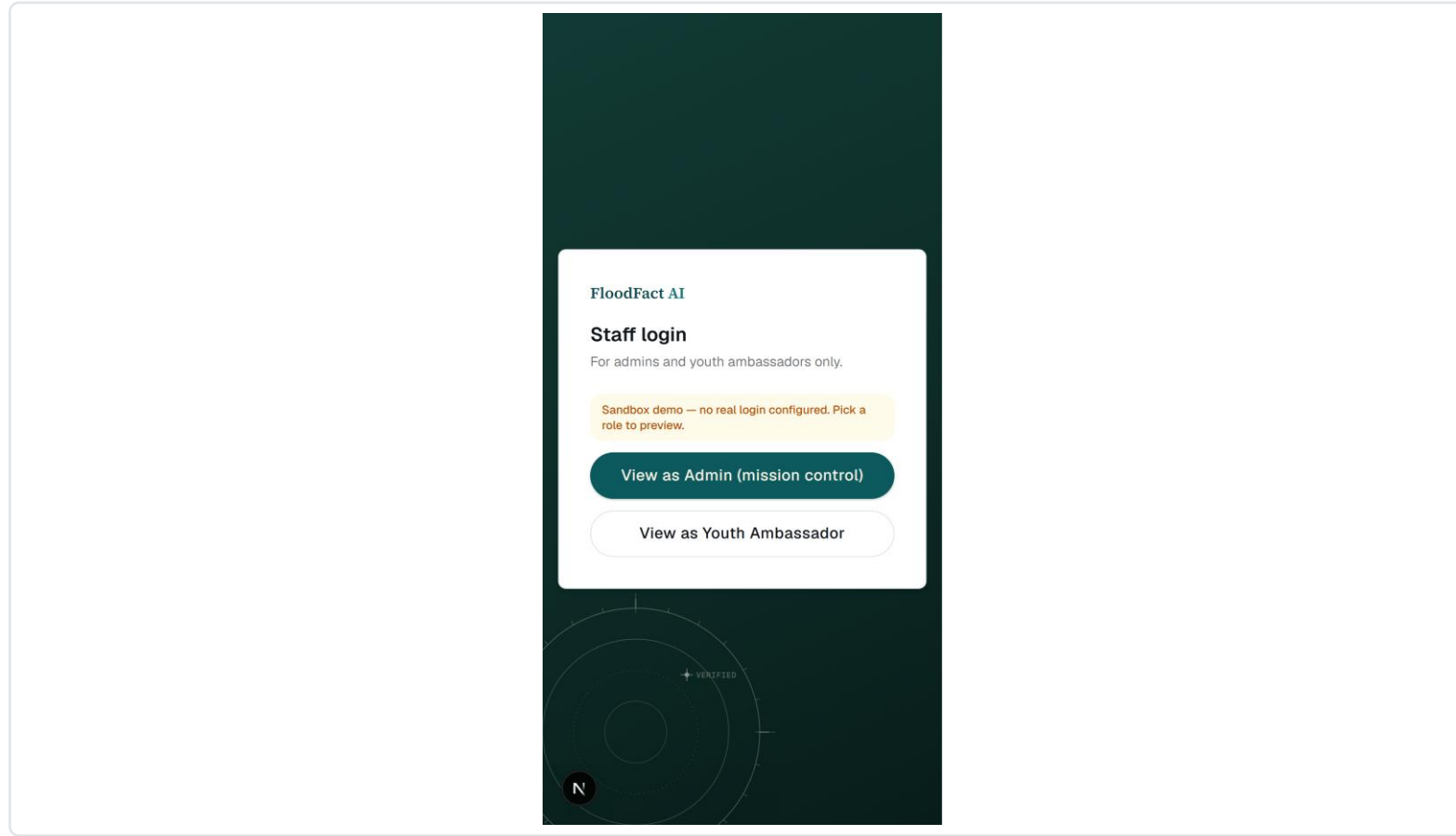
Per-provider sandbox mode

Independently, each external integration — WhatsApp, SMS, the LLM, embeddings, flood-risk — self-selects live or sandbox based on whether its own credential is present. Sandbox evidence is always stamped quality: "degraded," never "ok."

One deploy away from real

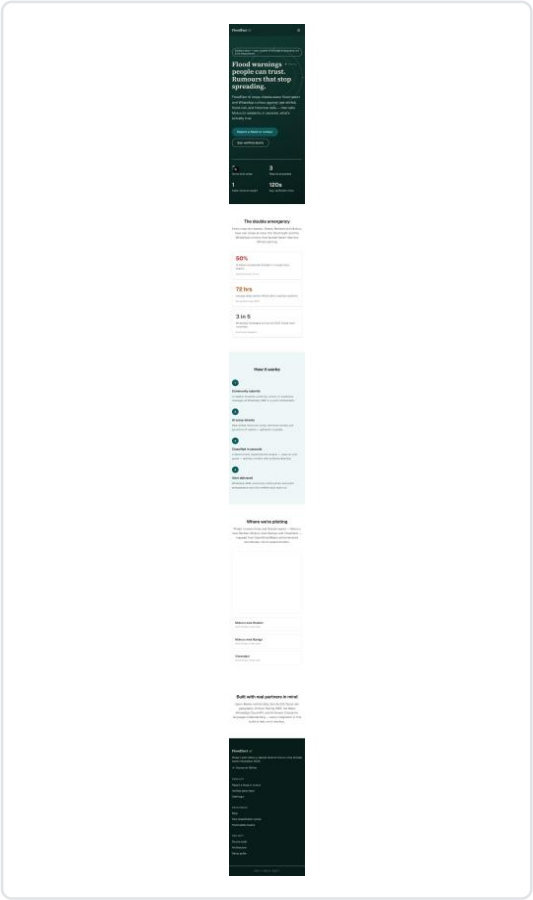
Add Supabase credentials and fixtures become live rows. Add one provider's API key and that one adapter goes live. Zero code changes either way — see docs/SETUP.md.

One login screen swaps in two real roles



Sandbox mode replaces Supabase Auth with a role switcher — Admin or Youth Ambassador — no accounts needed.

The public site states the deal in one screen



No install, no account — the landing page links straight into reporting and the public alerts feed.

“Water is rising fast near the railway crossing on Kanini Road”

FloodFact AI

Report a flood or a suspicious message

Tell us what you're seeing, or paste a message you're unsure about. We'll check it against current rainfall, flood-risk maps and historical records, and reply with a clear verdict — Verified Warning, Elevated Risk, or False Information.

What are you seeing or hearing? *

Water is rising fast near the railway crossing on Kanini Road

Where? (street, landmark, or sub-area)

Kanini Road, Mukuru kwa Reuben

Phone number (optional — to receive the verified result directly)

+254712345678

Submit report

WhatsApp or SMS? This same system checks messages sent via — the form is here for anyone who can't.

FloodFact AI

Phase 1 pilot: Mukuru, Nairobi. Built for the AI x City Climate Action Hackathon 2026.

[Source on GitHub](#)

PRODUCT

[Report a flood or rumour](#)

[Verified alerts feed](#)

[Staff login](#)

RESOURCES

[Blog](#)

[How classification works](#)

[Flood safety basics](#)

PROJECT

[Source code](#)

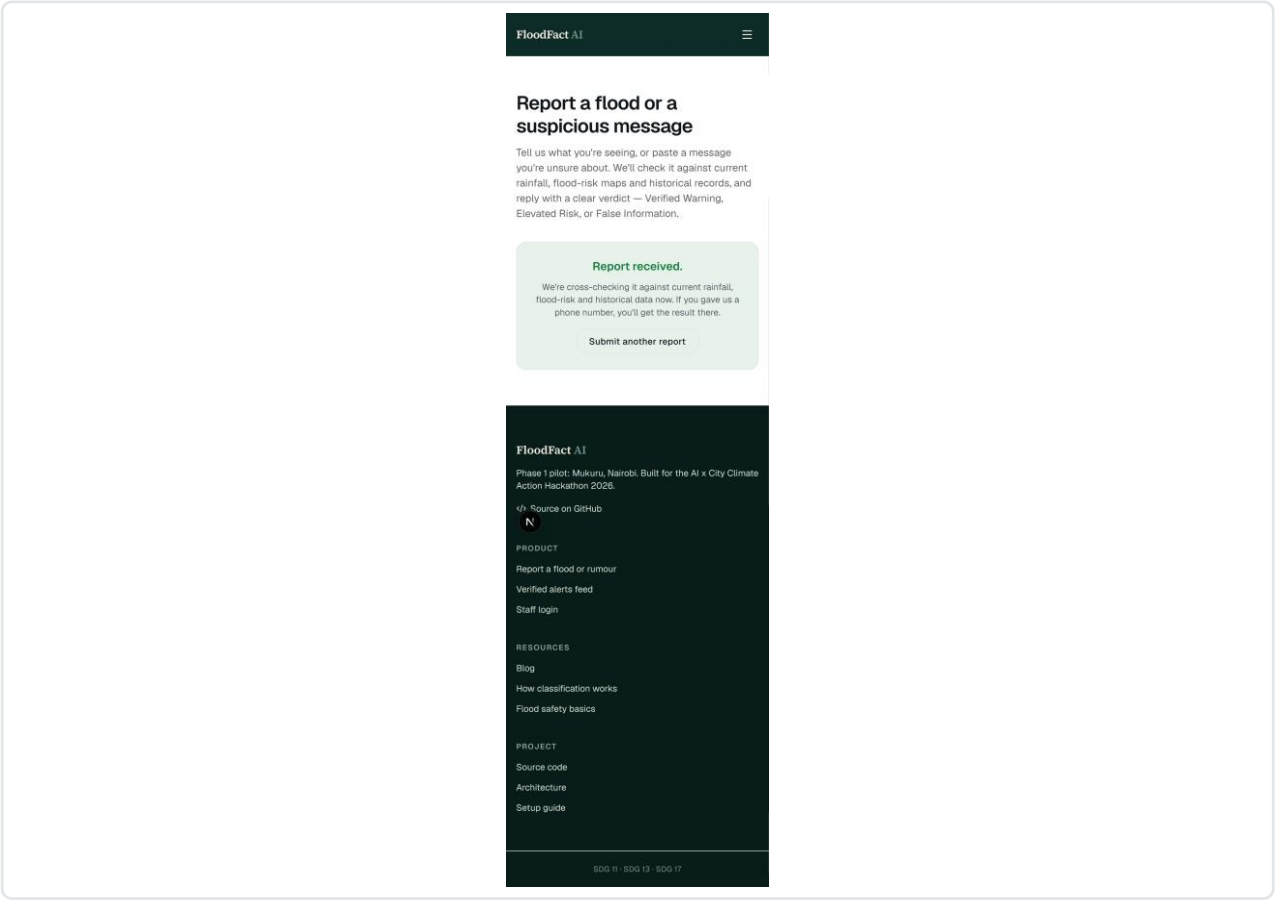
[Architecture](#)

[Setup guide](#)

806 R 800 I3 800 I7

This same Zod-validated schema also accepts inbound WhatsApp and SMS — the form is the fallback channel.

The resident is told their report is already moving



Submission triggers the classification pipeline immediately — no queue, no wondering if it arrived.

Every report runs the same six-stage, fully-audited pipeline

This is the real code path — `lib/pipeline/classify.ts` calling `lib/core/risk-engine.ts` — not a simplified diagram.

1

Understand

Claude extracts hazard type, claimed location, and urgency — logged verbatim to `nlu_extractions`.

2

Geocode

The claimed location is matched against a real Mukuru gazetteer and resolved to a pilot area via PostGIS.

3

Gather Evidence

Rainfall, flood-risk zone, historical base rate, corroboration, ambassador ground-truth, and rumor similarity — in parallel.

4

Decide

`risk-engine.ts` scores the evidence. Pure, deterministic, 23 unit tests. No LLM call in this step.

5

Explain

Claude writes the plain-language, bilingual EN/Swahili rationale for a verdict it did not make.

6

Alert

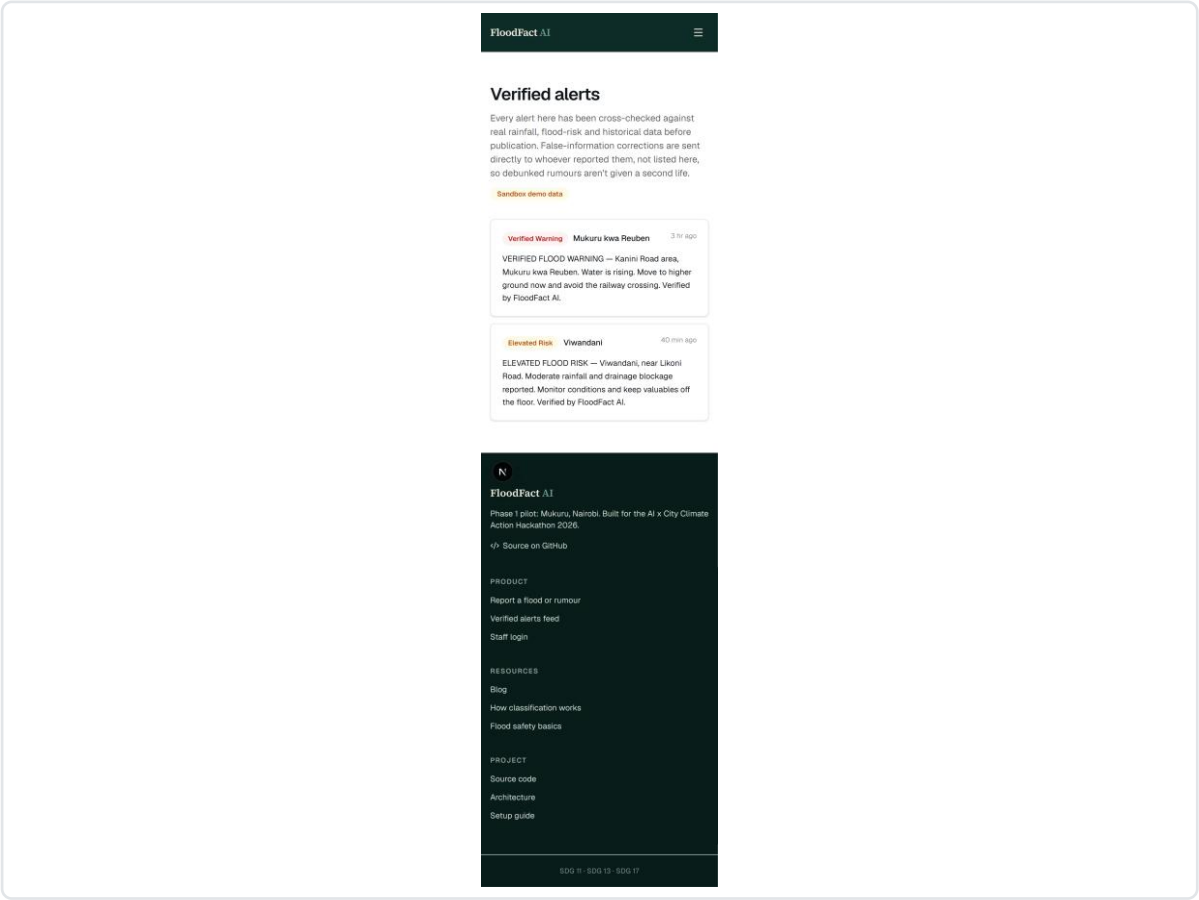
A channel-specific message goes out over WhatsApp, SMS, the public feed, and the ambassador queue.

A verdict that shows its work



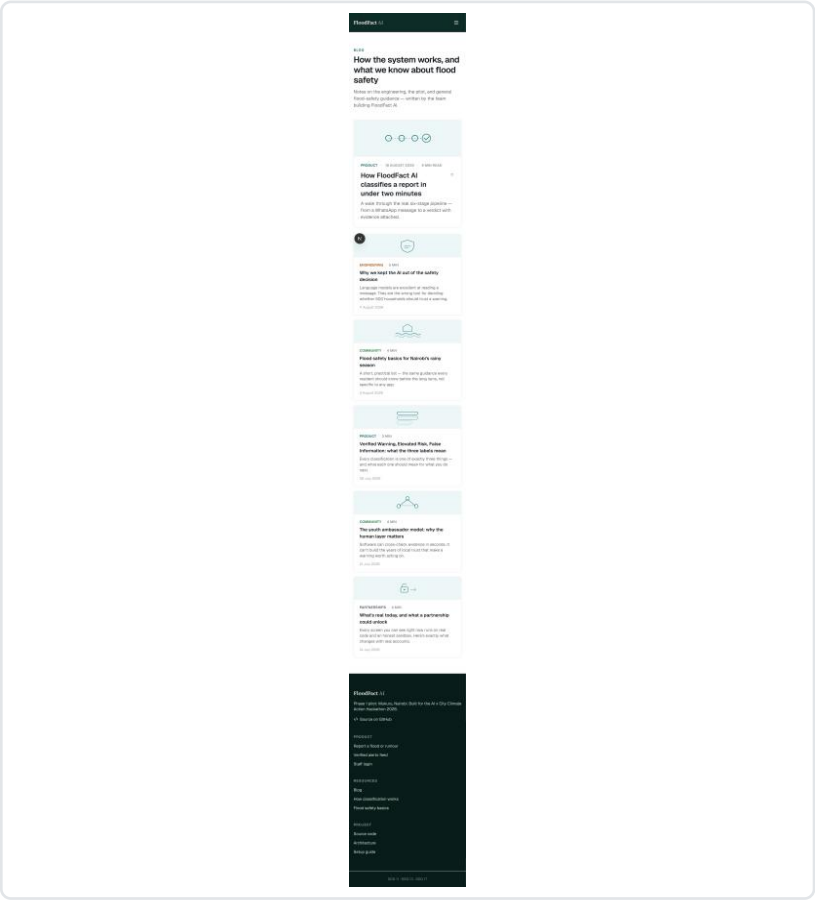
Verified Warning, a confidence score, a written rationale, and the evidence_items row behind each one.

Only verified and elevated-risk classifications appear here



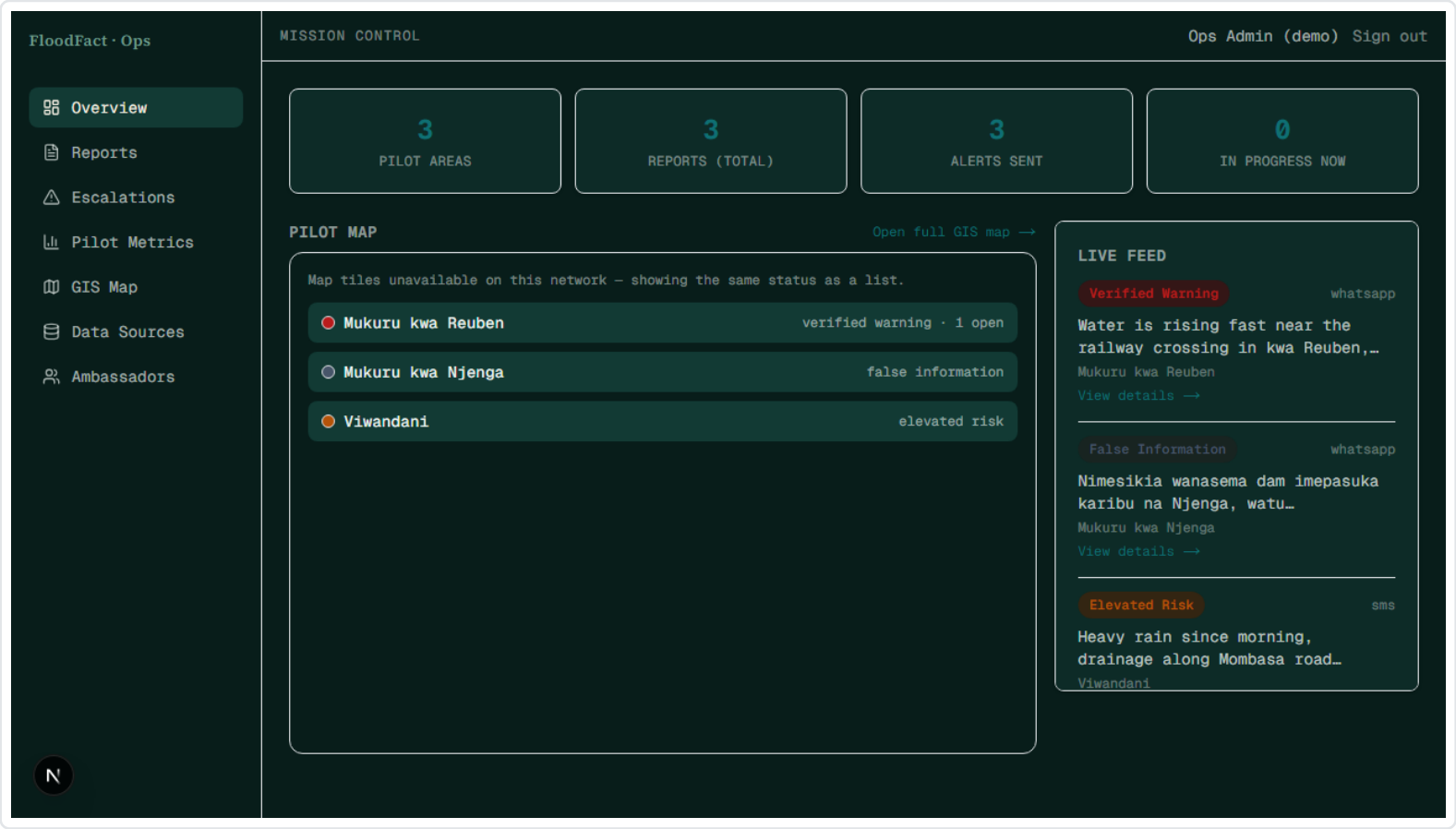
Reads from a dedicated public_alert_feed view — no raw reports, contact details, or open cases exposed.

A real blog — the engineering rationale, not marketing copy



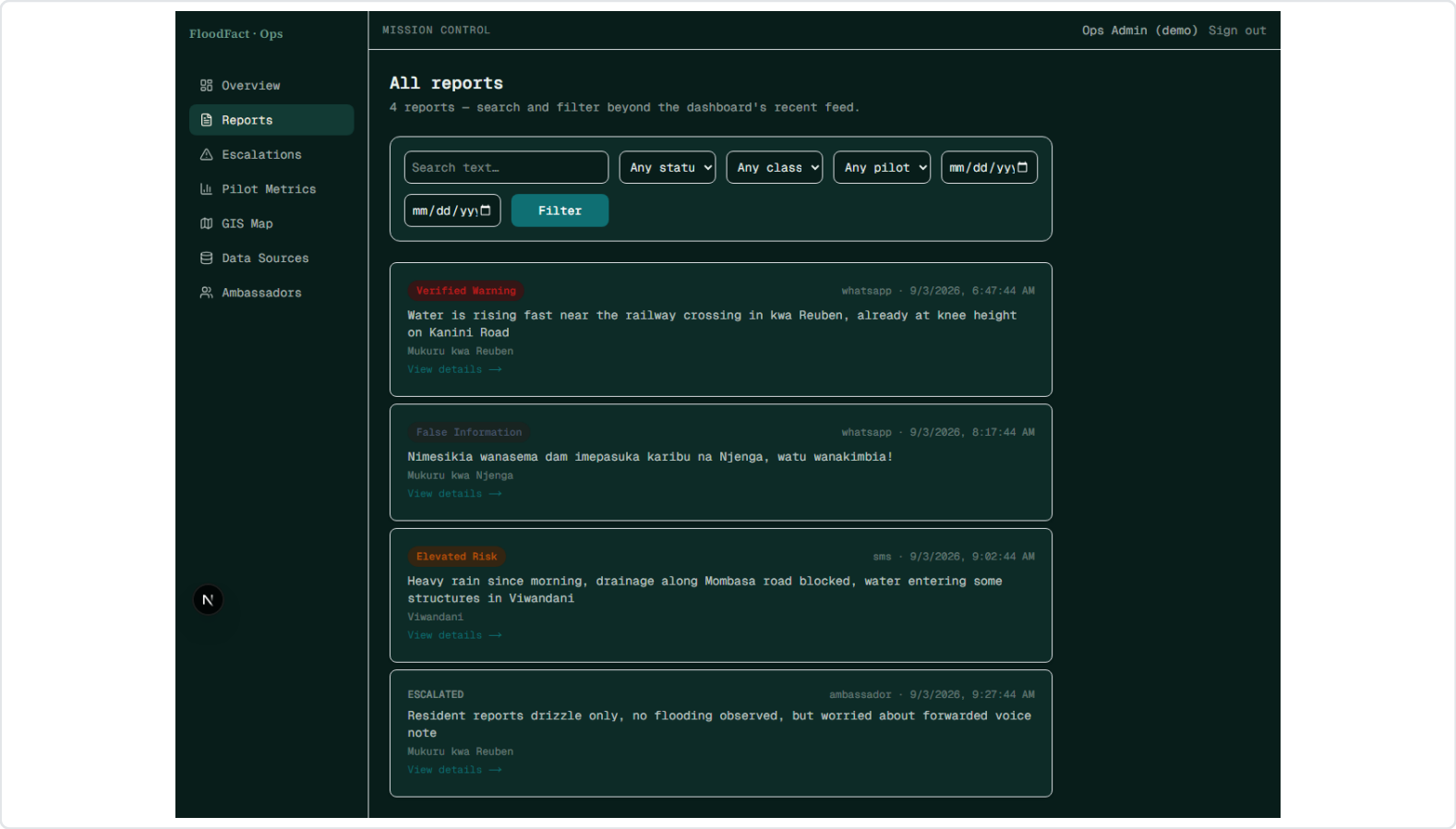
Six posts on how the pipeline works, why the AI doesn't decide, and general flood-safety guidance — no fabricated pilot results.

Every pilot area, one live map



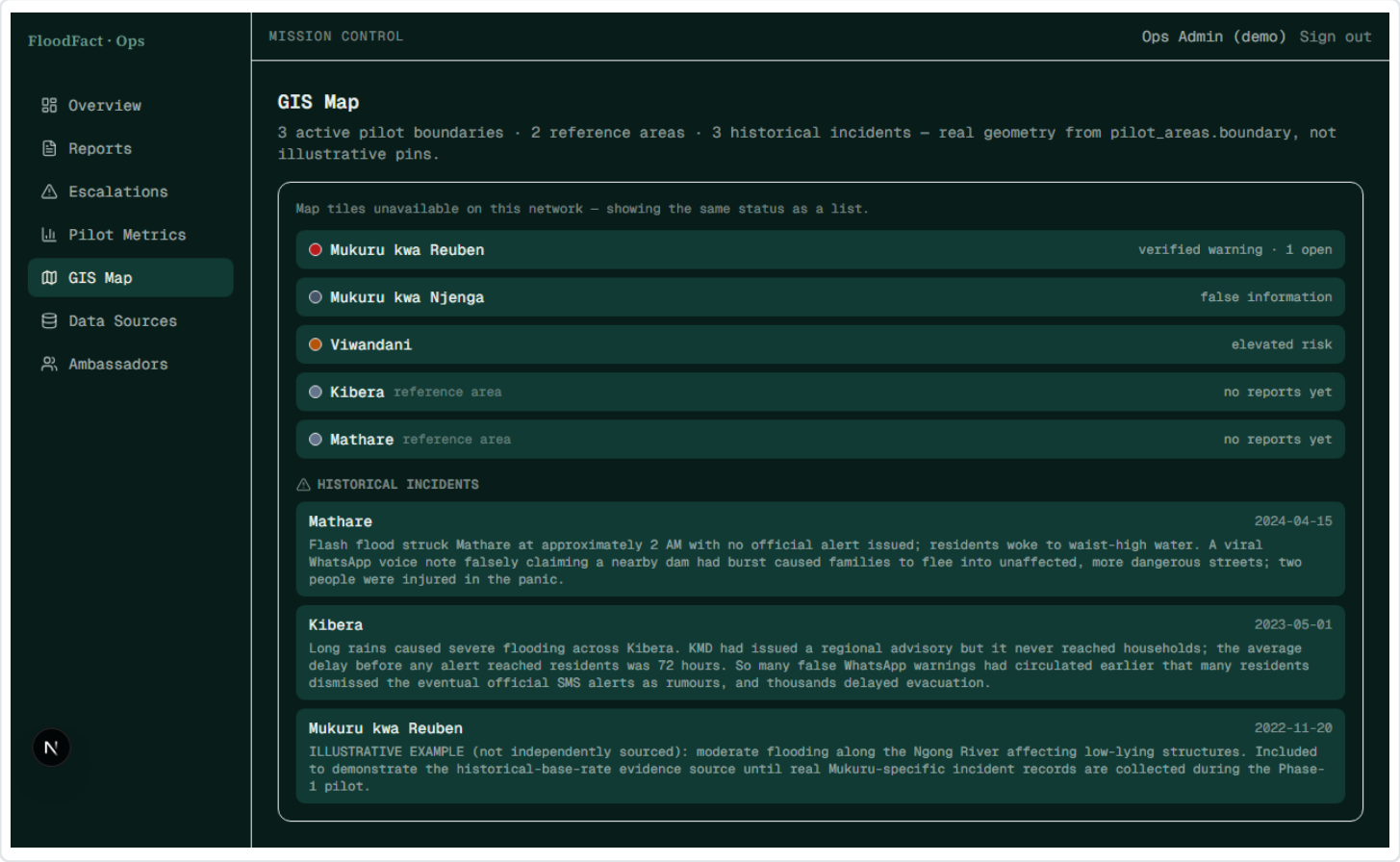
Mukuru's three wards, colored by latest verdict — falls back to a status list if tiles don't load.

Every report, searchable — not just the last 15



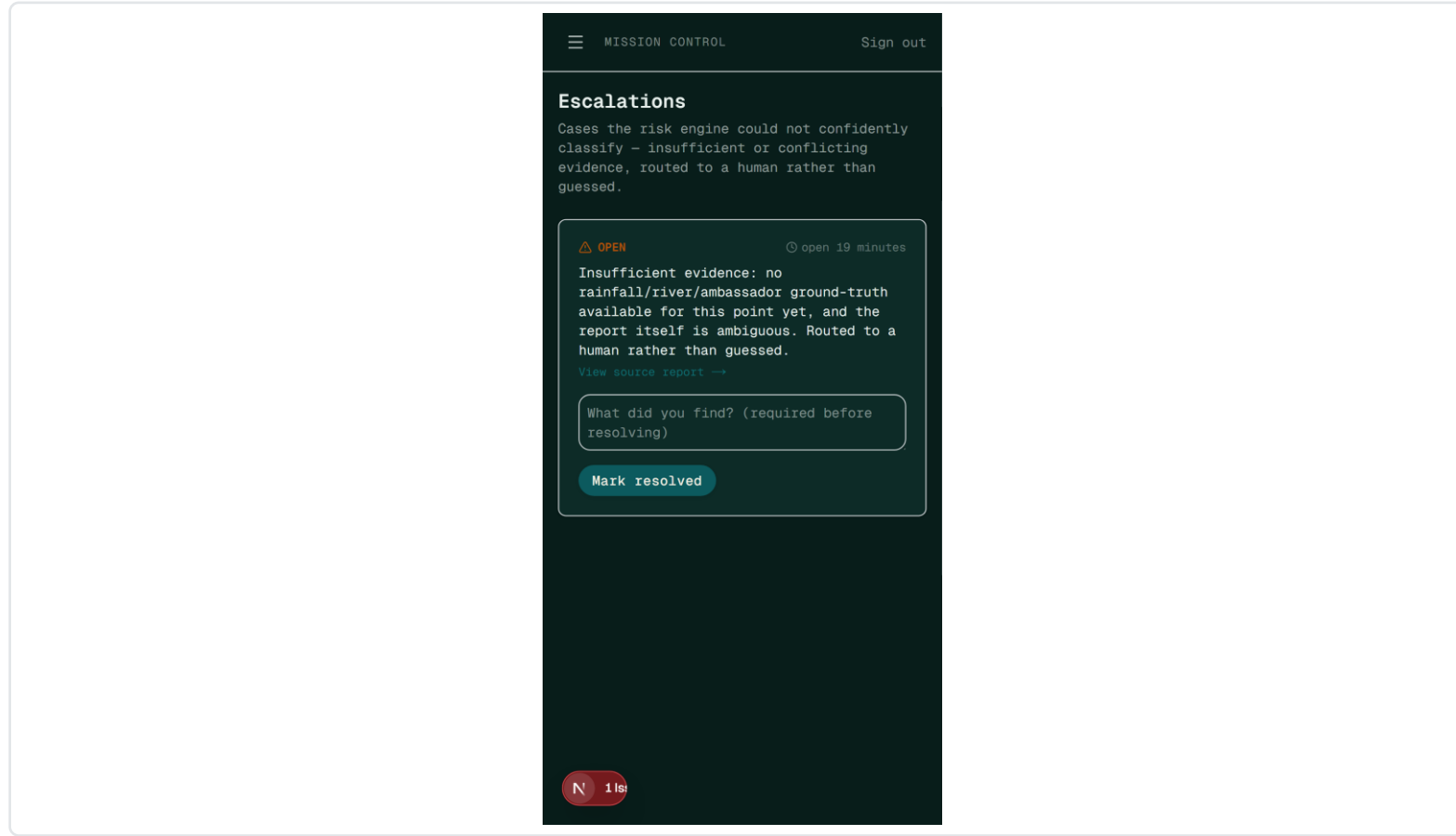
Filter by status, classification, pilot area, or date, and search report text directly — /admin/reports.

Pilot boundaries, reference wards, and historical incidents — one map



Real polygons from pilot_areas.boundary, plus cited historical floods — including an honestly-labeled illustrative one.

When evidence is insufficient, a human decides — never an algorithm alone



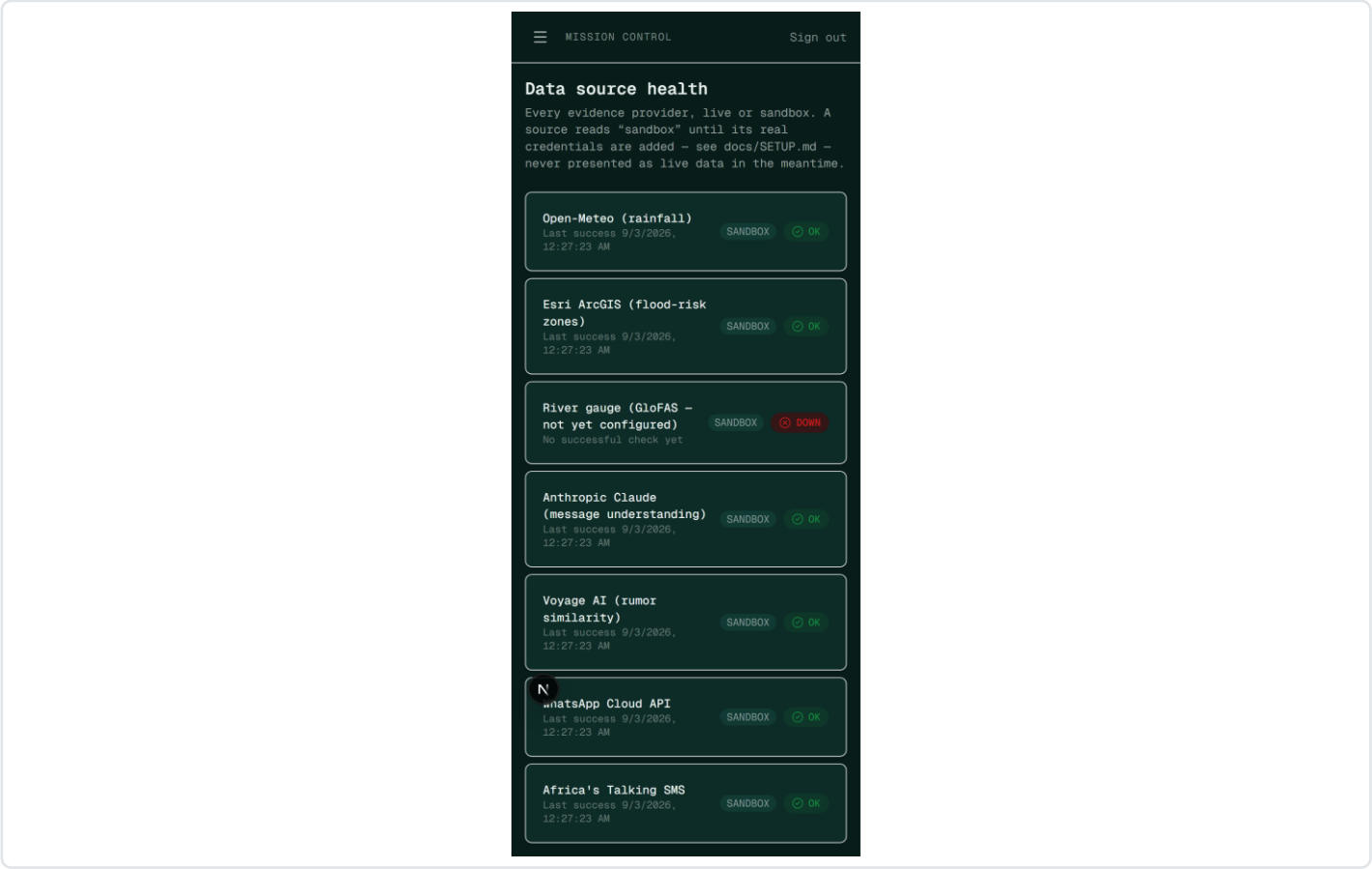
insufficient_evidence can never silently become "False Information" — it lands here for a human call, with a visible age badge and an hourly SLA-breach sweep so nothing goes stale unnoticed.

Pilot metrics computed from real database rows, not typed into a slide



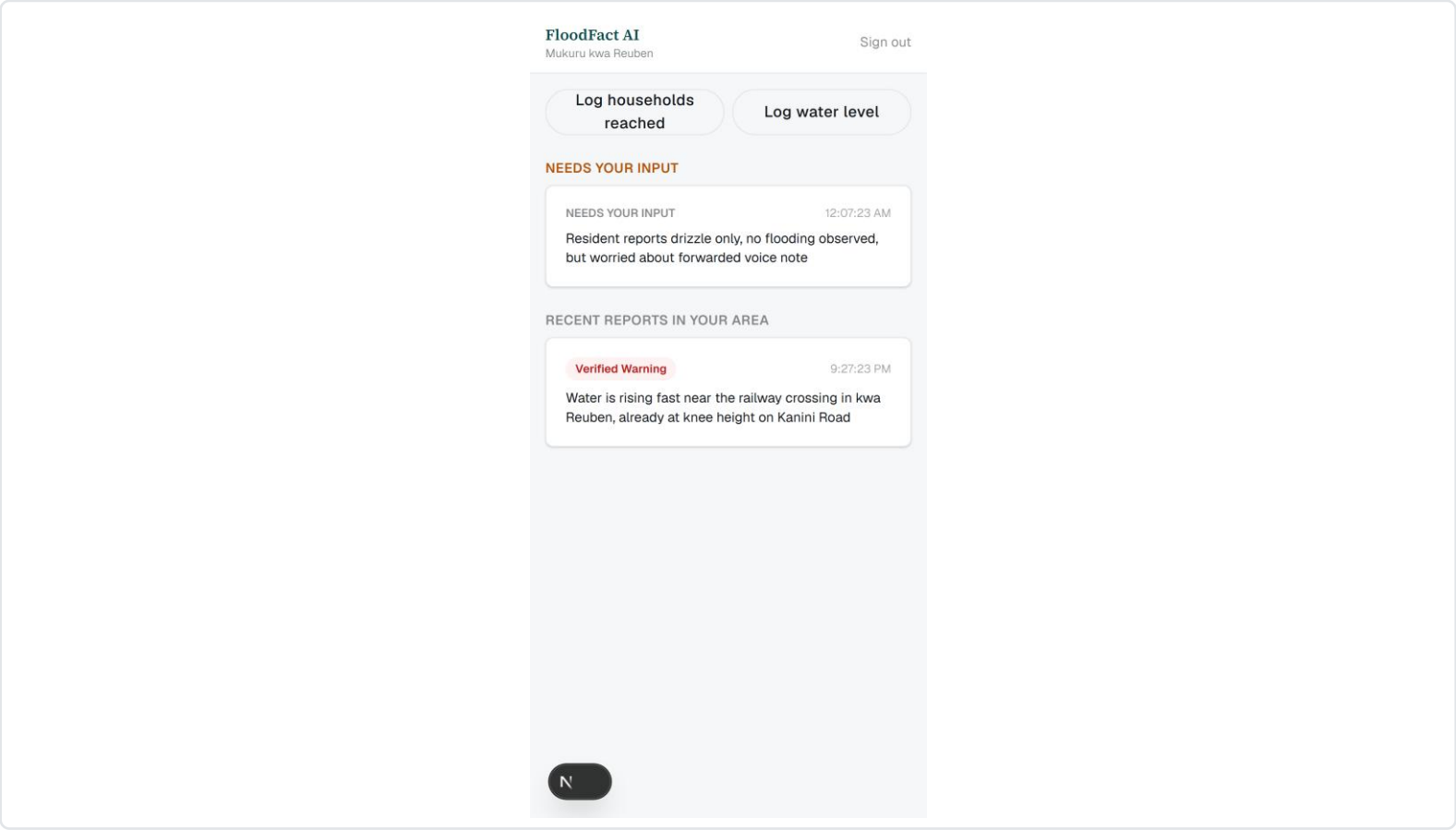
A live SQL view, not hand-typed numbers — recomputed from reports and ambassador observations.

Every data source's health, live or sandbox, in plain sight



Each provider's live/sandbox mode and status, visible to the people operating the system.

Youth ambassadors see only their own ward's queue



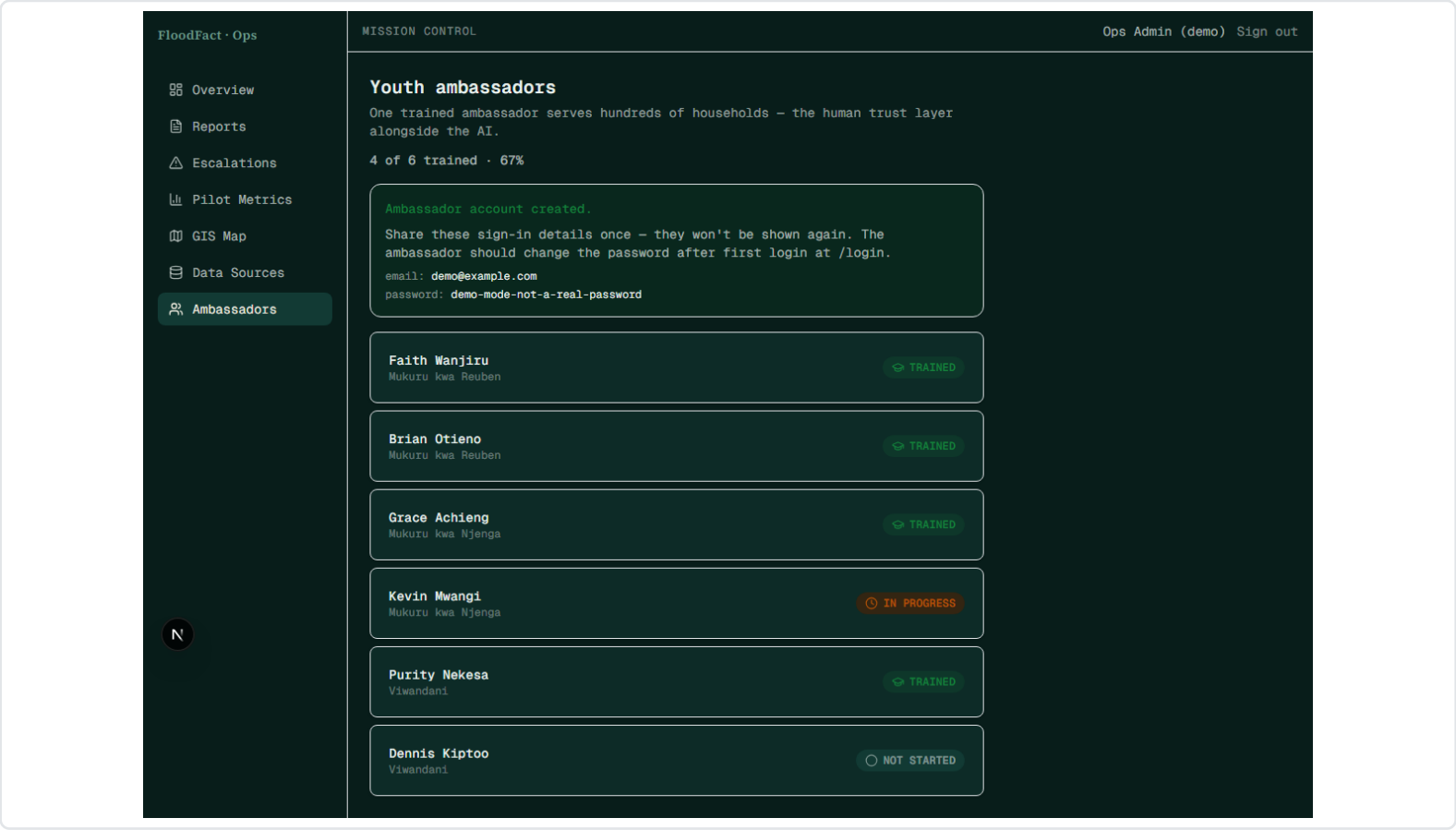
Row-level security enforces this at the database — an ambassador's session cannot query another ward.

A visual confirmation and a water-level reading feed straight back into the evidence store

The screenshot displays the FloodFact AI mobile application interface. At the top, the header includes the app name 'FloodFact AI' and the user's name 'Mukuru kwa Reuben', with a 'Sign out' link on the right. Below the header, there is a navigation bar with a back arrow and the text 'Back to queue'. The main content area shows a timestamp '9/2/2026, 9:27:23 PM' followed by a report title: 'Water is rising fast near the railway crossing in kwa Reuben, already at knee height on Kanini Road'. Below the title, the location 'Mukuru kwa Reuben' is listed. A 'Verified Warning' section contains a paragraph: 'Heavy rainfall in the last 3 hours, the location sits inside a mapped high flood-risk zone, similar events have happened here before, and 2 other nearby reports corroborate rising water.' Below this, a confirmation prompt asks 'Did you check this in person?'. There are two buttons: a red button labeled 'Yes, flooding confirmed' and a white button labeled 'No flooding seen'. At the bottom left, there is a red circular button with a white 'N' and the text '1 Issue'.

A visual confirmation and a water reading feed back in as evidence for the next nearby report.

Adding an ambassador is one form, not a database console



Creates the Auth login, the ambassadors row, and the user_roles row in one action — admin-only.

Every integration switches independently — no code changes either way

Integration	Right now (zero paid keys)	Our real pilot deployment, right now
Rainfall & forecast	Open-Meteo — the real public API, already live (keyless)	No key needed — already real
Flood-risk zones	Seeded PostGIS polygon for Mukuru	Live — WRI Aqueduct riverine flood-risk atlas (ArcGIS)
WhatsApp intake/alerts	Deterministic sandbox responder	Not yet — needs a Meta developer app + token
SMS intake/alerts	Deterministic sandbox responder	Live — Africa's Talking sandbox account, real API calls
Claim extraction + rationale	Deterministic template text	Live — Anthropic Claude, verified
Rumor-pattern similarity	Local hash embedding	Live — Voyage AI embeddings, verified
Database, Auth, this deck	Static fixtures — DEMO_MODE, no Supabase project	Live — real Supabase project, 9 migrations, RLS everywhere

Each row switches on its own credential's presence, not a global flag. Exact steps for every row: docs/SETUP.md.

The whole system is one clone away

Read the code

Full source, 6 SQL migrations with RLS, and 43 passing unit tests — 23 of them exhaustively covering the risk engine's safety rules.

Run it yourself

npm install && npm run dev boots the entire product — public site, admin console, ambassador PWA — from seed fixtures. No accounts, no keys.

Go live

docs/SETUP.md walks through provisioning Supabase, Meta, Africa's Talking, Anthropic, and Voyage, step by step.

THANK YOU

**This is not a concept.
It is a working system — already live on 4 of 5 real
integrations.**

Concept: FloodFact-AI-Concept.pdf · Pilot Plan: FloodFact-AI-Pilot-Plan.pdf · Repository:
github.com/VinceOdundo/FloodfactAI

Team: Samuel Mburu Kairu · Vincent Odundo · Alvin Nyota · Amani Maria — Nairobi, Kenya